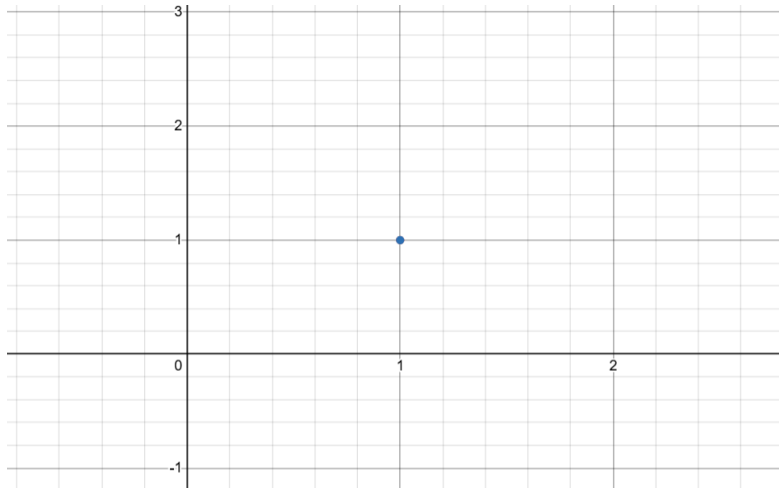


Test of Point.intersect(Point)

Case 1 and 2:



```
Point p1 = new Point(1, 1);
```

```
Point p2 = new Point(1, 1);
```

```
assertTrue(p1.intersect(p1), "A point should intersect with itself.");
```

```
assertTrue(p1.intersect(p2), "Point p1 should intersect with Point p2.");
```

```
assertTrue(p2.intersect(p1), "Point p2 should intersect with Point p1.");
```

Expected result: true

Code result: true

Test result: pass

Case 3:



```
Point p1 = new Point(1, 1);
```

```
Point p3 = new Point(2, 2);
```

```
assertFalse(p1.intersect(p3), "Point p1 should not intersect with Point p3.");
```

```
assertFalse(p3.intersect(p1), "Point p3 should not intersect with Point p1.");
```

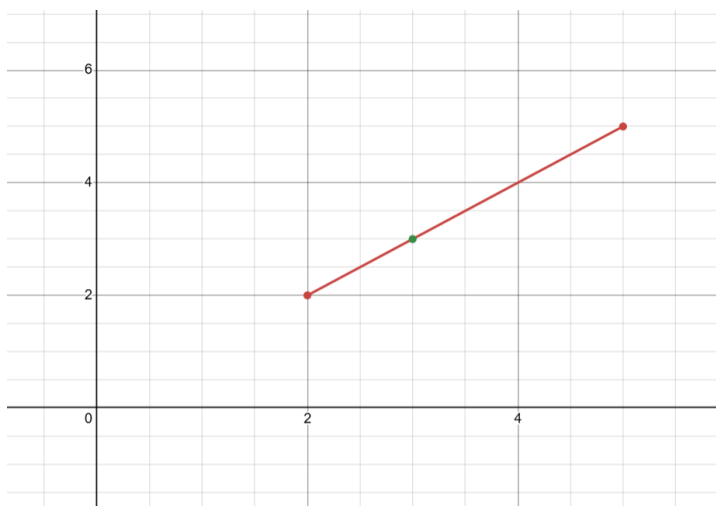
Expected result: false

Code result: false

Test result: pass

Test of Point.intersect(LineSeg)

Case 1:



```
Point p1 = new Point(2, 2);  
Point p2 = new Point(5, 5);  
LineSeg l1 = new LineSeg(p1, p2, false);  
Point p3 = new Point(3, 3);  
assertTrue(p3.intersect(l1), "Point p3 should intersect with LineSeg l1.");  
assertTrue(l1.intersect(p3), "LineSeg l1 should intersect with Point p3.");
```

Expected result: true

Code result: true

Test result: pass

Case 2:



```
Point p1 = new Point(2, 2);  
Point p2 = new Point(5, 5);  
LineSeg l1 = new LineSeg(p1, p2, false);  
Point p4 = new Point(6, 6);  
assertFalse(p4.intersect(l1), "Point p4 should not intersect with LineSeg l1.");  
assertFalse(l1.intersect(p4), "LineSeg l1 should not intersect with Point p4.");
```

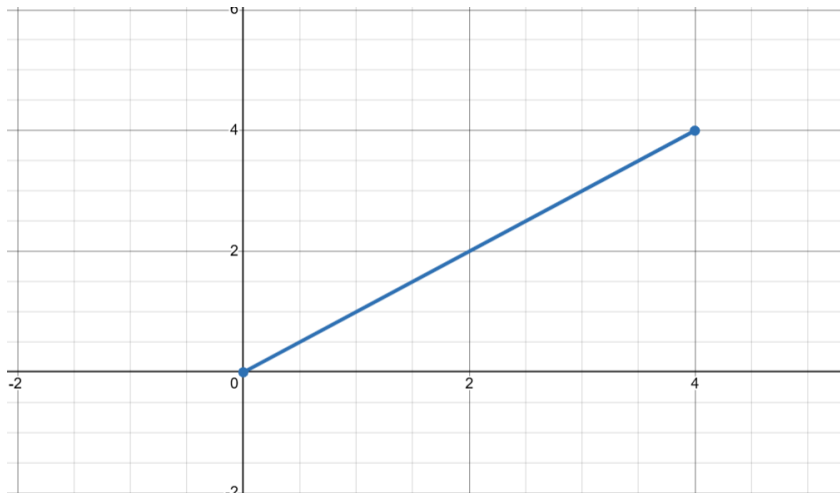
Expected result: false

Code result: false

Test result: pass

Test of LineSeg.intersect(LineSeg)

Case 1:



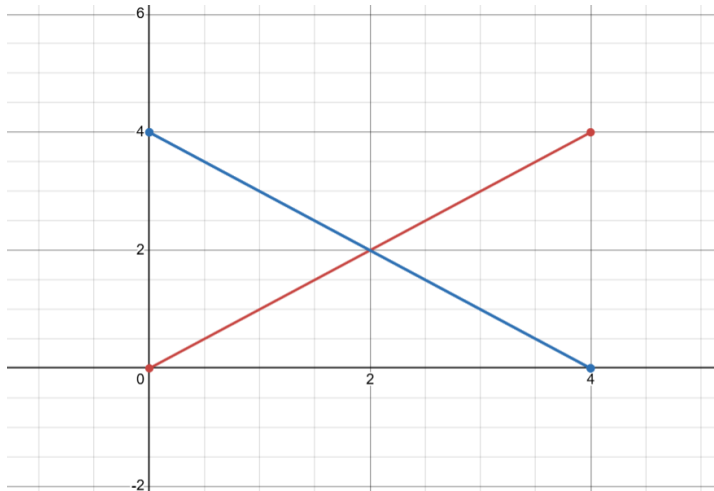
```
LineSeg l1 = new LineSeg(new Point(0, 0), new Point(4, 4), false);  
assertFalse(l1.intersect(l1), "A LineSeg shouldn't intersect with itself.");
```

Expected result: false

Code result: false

Test result: pass

Case 2:



```
LineSeg l1 = new LineSeg(new Point(0, 0), new Point(4, 4), false);
```

```
LineSeg l2 = new LineSeg(new Point(4, 0), new Point(0, 4), false);
```

```
assertTrue(l1.intersect(l2), "LineSeg l1 should intersect with LineSeg l2.");
```

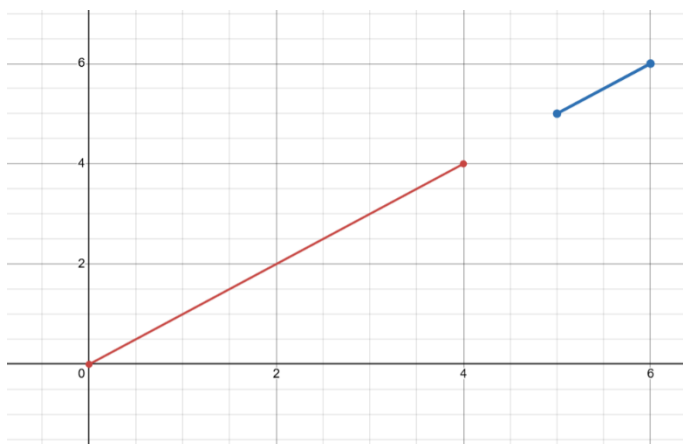
```
assertTrue(l2.intersect(l1), "LineSeg l2 should intersect with LineSeg l1.");
```

Expected result: true

Code result: true

Test result: pass

Case 3:



```
LineSeg l1 = new LineSeg(new Point(0, 0), new Point(4, 4), false);
```

```
LineSeg l3 = new LineSeg(new Point(5, 5), new Point(6, 6), false);
```

```
assertFalse(l1.intersect(l3), "LineSeg l1 should not intersect with LineSeg l3.");
```

```
assertFalse(l3.intersect(l1), "LineSeg l3 should not intersect with LineSeg l1.");
```

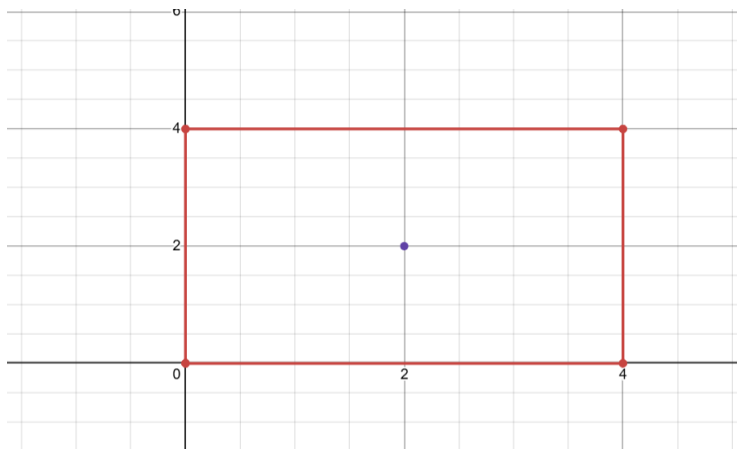
Expected result: false

Code result: false

Test result: pass

Test of `Point.intersect(Rectangle)`

Case 1:



```
Rectangle r1 = new Rectangle(new Point(0, 0), new Point(0, 4), new Point(4, 4), new Point(4, 0));
```

```
Point p1 = new Point(2, 2);
```

```
assertTrue(p1.intersect(r1), "Point p1 should intersect with Rectangle r1.");
```

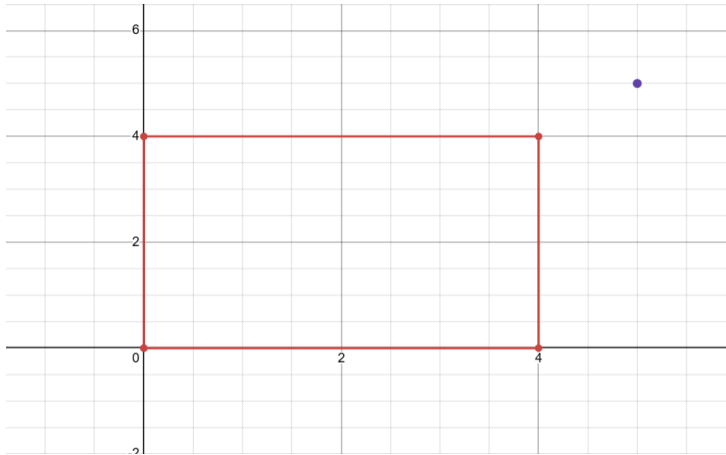
```
assertTrue(r1.intersect(p1), "Rectangle r1 should intersect with Point p1.");
```

Expected result: true

Code result: true

Test result: pass

Case 2:



```
Rectangle r1 = new Rectangle(new Point(0, 0), new Point(0, 4), new Point(4, 4), new
Point(4, 0));
```

```
Point p2 = new Point(5, 5);
```

```
assertFalse(p2.intersect(r1), "Point p2 should not intersect with Rectangle r1.");
```

```
assertFalse(r1.intersect(p2), "Rectangle r1 should not intersect with Point p2.");
```

Expected result: false

Code result: false

Test result: pass

Test of LineSeg.intersect(Rectangle)

Case 1:



```
Rectangle r1 = new Rectangle(new Point(0, 0), new Point(0, 4), new Point(4, 4), new
Point(4, 0));
```

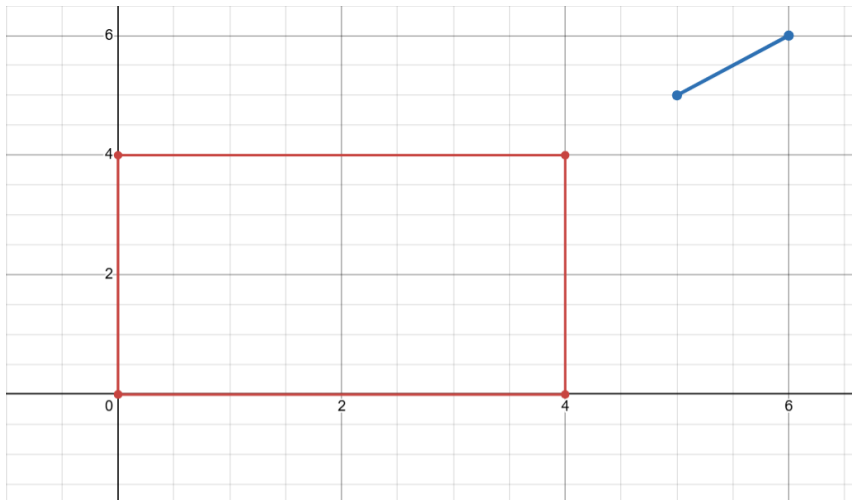
```
LineSeg l1 = new LineSeg(new Point(-1, 1), new Point(1, -1), false);  
  
assertTrue(l1.intersect(r1), "LineSeg l1 should intersect with Rectangle r1.");  
  
assertTrue(r1.intersect(l1), "Rectangle r1 should intersect with LineSeg l1.");
```

Expected result: true

Code result: true

Test result: pass

Case 2:



```
Rectangle r1 = new Rectangle(new Point(0, 0), new Point(0, 4), new Point(4, 4), new  
Point(4, 0));
```

```
LineSeg l2 = new LineSeg(new Point(5, 5), new Point(6, 6), false);  
  
assertFalse(l2.intersect(r1), "LineSeg l2 should not intersect with Rectangle r1.");  
  
assertFalse(r1.intersect(l2), "Rectangle r1 should not intersect with LineSeg l2.");
```

Expected result: false

Code result: false

Test result: pass

Test of Rectangle.intersect(Rectangle)

Case 1:



```
Rectangle r1 = new Rectangle(new Point(0, 0), new Point(0, 4), new Point(4, 4), new Point(4, 0));
```

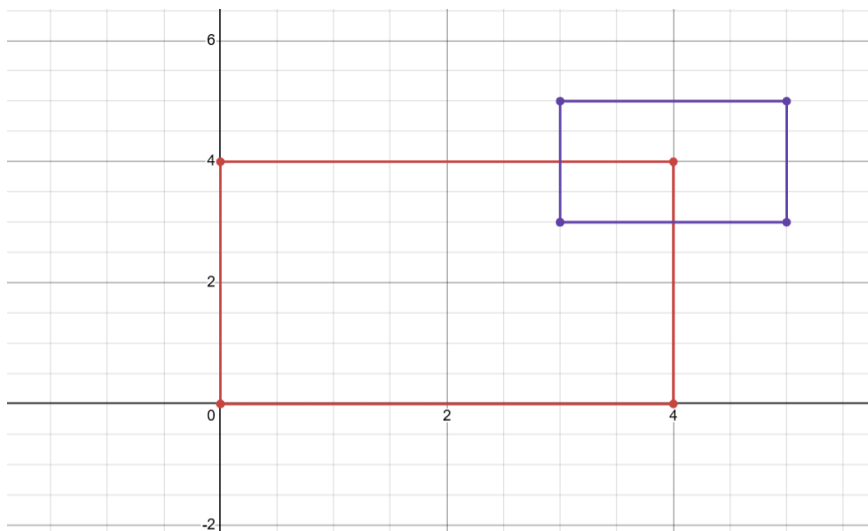
```
assertTrue(r1.intersect(r1), "A rectangle should intersect with itself.");
```

Expected result: true

Code result: true

Test result: pass

Case 2:



```
Rectangle r1 = new Rectangle(new Point(0, 0), new Point(0, 4), new Point(4, 4), new Point(4, 0));
```

```
Rectangle r2 = new Rectangle(new Point(3, 3), new Point(3, 5), new Point(5, 5), new
Point(5, 3));
```

```
assertTrue(r1.intersect(r2), "Rectangle r1 should intersect with Rectangle r2.");
```

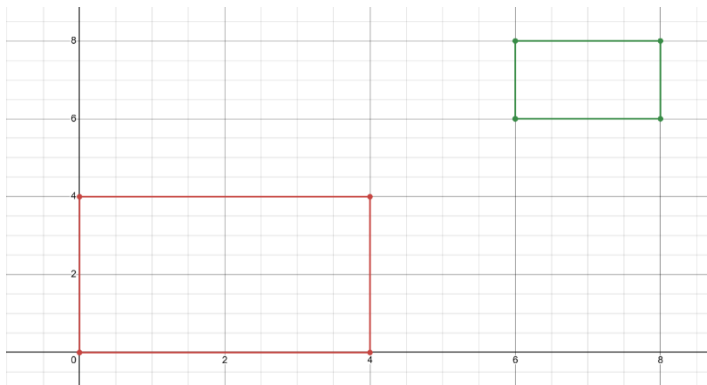
```
assertTrue(r2.intersect(r1), "Rectangle r2 should intersect with Rectangle r1.");
```

Expected result: true

Code result: true

Test result: pass

Case 3:



```
Rectangle r1 = new Rectangle(new Point(0, 0), new Point(0, 4), new Point(4, 4), new
Point(4, 0));
```

```
Rectangle r3 = new Rectangle(new Point(6, 6), new Point(6, 8), new Point(8, 8), new
Point(8, 6));
```

```
assertFalse(r1.intersect(r3), "Rectangle r1 should not intersect with Rectangle r3.");
```

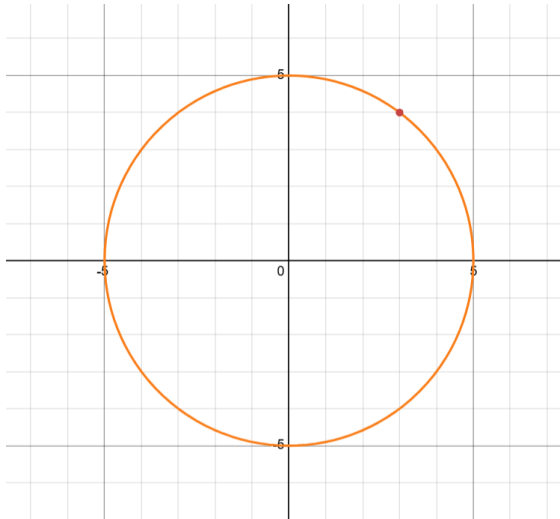
```
assertFalse(r3.intersect(r1), "Rectangle r3 should not intersect with Rectangle r1.");
```

Expected result: false

Code result: false

Test result: pass

Test of Point.intersect(Circle)



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

```
Point p1 = new Point(3, 4);
```

```
assertTrue(p1.intersect(c1), "Point p1 should intersect with Circle c1.");
```

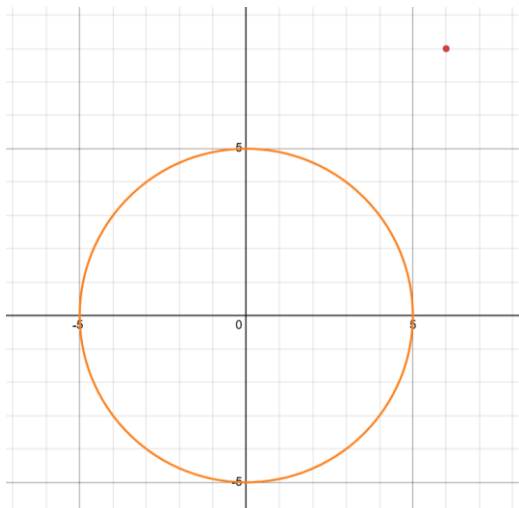
```
assertTrue(c1.intersect(p1), "Circle c1 should intersect with Point p1.");
```

Expected result: true

Code result: true

Test result: pass

Case 2:



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

```
Point p2 = new Point(6, 8);
```

```
assertFalse(p2.intersect(c1), "Point p2 should not intersect with Circle c1.");
```

```
assertFalse(c1.intersect(p2), "Circle c1 should not intersect with Point p2.");
```

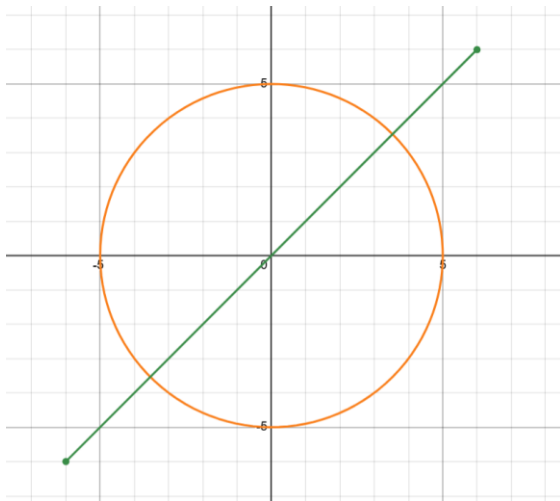
Expected result: false

Code result: false

Test result: pass

Test of LineSeg.intersect(Circle)

Case 1:



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

```
LineSeg l1 = new LineSeg(new Point(-6, -6), new Point(6, 6), false);
```

```
assertTrue(l1.intersect(c1), "LineSeg l1 should intersect with Circle c1.");
```

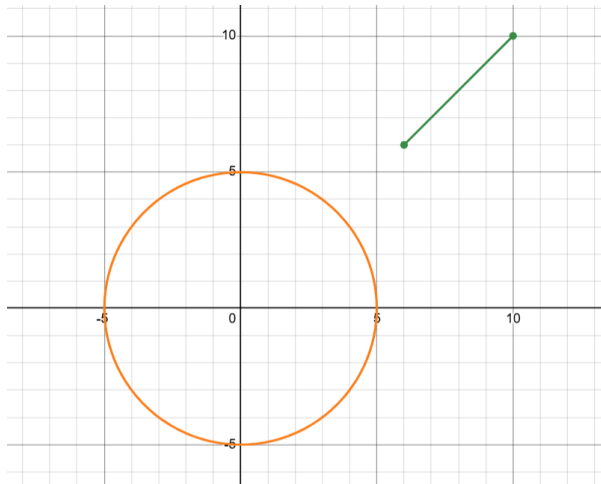
```
assertTrue(c1.intersect(l1), "Circle c1 should intersect with LineSeg l1.");
```

Expected result: true

Code result: true

Test result: pass

Case 2:



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

```
LineSeg l2 = new LineSeg(new Point(6, 6), new Point(10, 10), false);
```

```
assertFalse(l2.intersect(c1), "LineSeg l2 should not intersect with Circle c1.");
```

```
assertFalse(c1.intersect(l2), "Circle c1 should not intersect with LineSeg l2.");
```

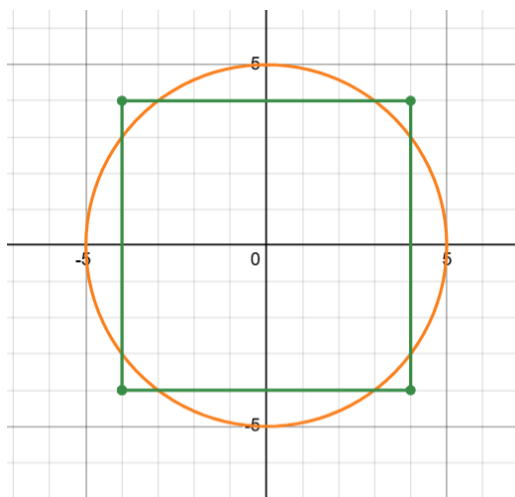
Expected result: false

Code result: false

Test result: pass

Test of Rectangle.intersect(Circle)

Case 1:



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

```
Rectangle r1 = new Rectangle(new Point(-4, -4), new Point(-4, 4), new Point(4, 4), new Point(4, -4));
```

```
assertTrue(r1.intersect(c1), "Rectangle r1 should intersect with Circle c1.");
```

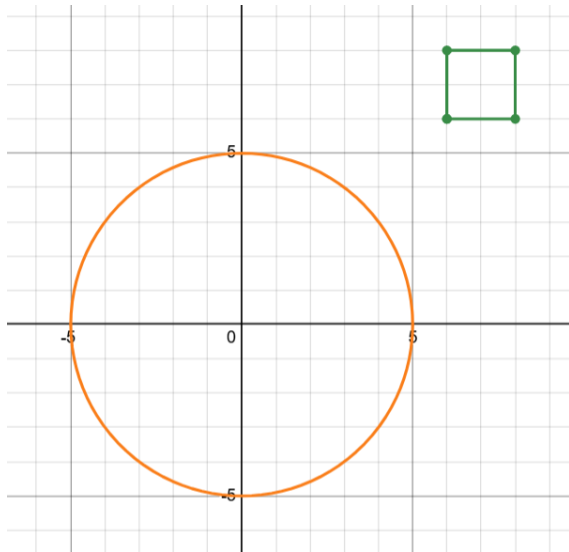
```
assertTrue(c1.intersect(r1), "Circle c1 should intersect with Rectangle r1.");
```

Expected result: true

Code result: true

Test result: pass

Case 2:



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

```
Rectangle r2 = new Rectangle(new Point(6, 6), new Point(6, 8), new Point(8, 8), new Point(8, 6));
```

```
assertFalse(r2.intersect(c1), "Rectangle r2 should not intersect with Circle c1.");
```

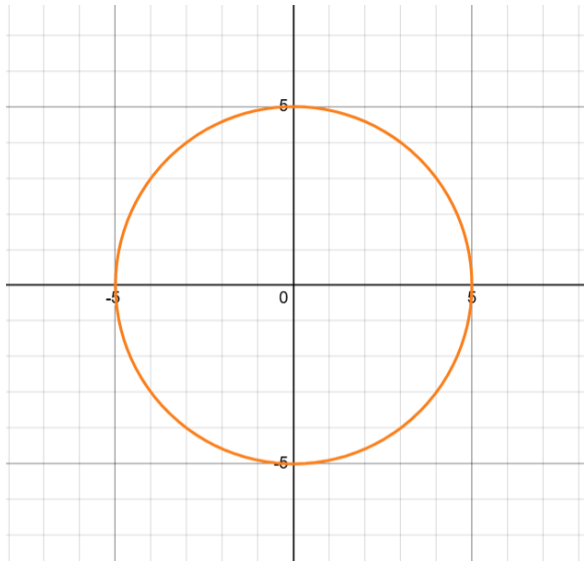
```
assertFalse(c1.intersect(r2), "Circle c1 should not intersect with Rectangle r2.");
```

Expected result: false

Code result: false

Test result: pass

Test of Circle.intersect(Circle)



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

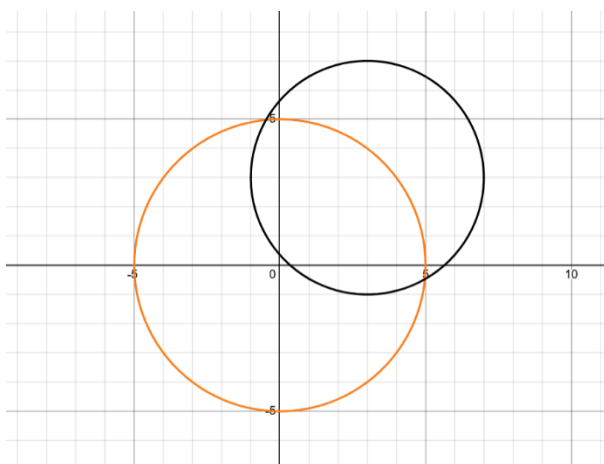
```
assertTrue(c1.intersect(c1), "A circle should intersect with itself.");
```

Expected result: true

Code result: true

Test result: pass

Case 2:



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

```
Circle c2 = new Circle(new Point(3, 3), 4);
```

```
assertTrue(c1.intersect(c2), "Circle c1 should intersect with Circle c2.");
```

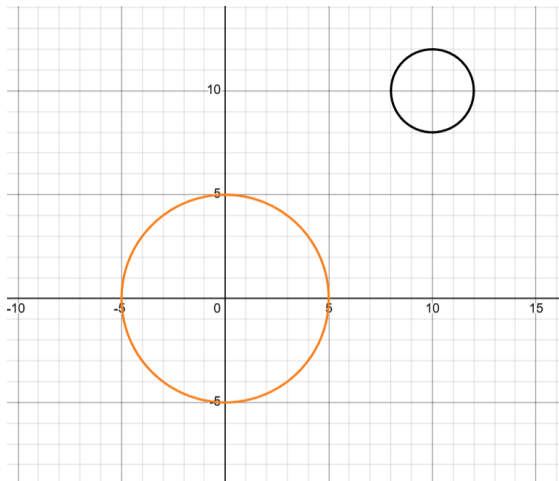
```
assertTrue(c2.intersect(c1), "Circle c2 should intersect with Circle c1.");
```

Expected result: true

Code result: true

Test result: pass

Case 3:



```
Circle c1 = new Circle(new Point(0, 0), 5);
```

```
Circle c3 = new Circle(new Point(10, 10), 2);
```

```
assertFalse(c1.intersect(c3), "Circle c1 should not intersect with Circle c3.");
```

```
assertFalse(c3.intersect(c1), "Circle c3 should not intersect with Circle c1.");
```

Expected result: false

Code result: false

Test result: pass